this is the cpp stl topic ...Sequence Containers (vector, deque, list, forward_list, array)

Associative Containers (set, multiset, map, multimap)

Unordered Containers (unordered_set, unordered_multiset, unordered_map, unordered_multimap)

Iterators (types and traversal functions)

Algorithms (sort, search, transform, etc.)

Utility Components (pair, tuple, swap, move, etc.)

Functors / Function Objects

Lambda Expressions

Container Adaptors (stack, queue, priority_queue)

Iterator Adaptors (back_inserter, stream iterators, etc.)

Allocators (advanced memory customization) .....write very details each and every possible operation with the formulae that is supported by the same for eg. insert,delete etc.. alos expalin the function of operation ..the answer can be of many pages

Great — this is one of the **most important topics in C++ STL**, and you're asking exactly the right question 👍
Below is a **structured, exhaustive + beginner-friendly → professional-level guide** covering:

- every container type
- **all commonly-used functions & operations**
- semantics
- complexity notes
- example "formulae" (usage patterns)

This **reads like a mini-book / cheat-sheet**, but explained clearly so you can revise easily.

⭐ MASTER GUIDE TO C++ STL

(Containers • Iterators • Algorithms • Utilities • Adaptors • Lambdas • Functors)

1️⃣ SEQUENCE CONTAINERS

> Maintain order of elements.

Includes

`vector` , `deque` , `list` , `forward_list` , `array`

🟦 A. vector<T>

Dynamic array — **contiguous memory**.

🔹 Declaration

```cpp
vector<int> v;
vector<int> v(n);          // n default elements
vector<int> v(n, 5);       // n elements = 5
vector<int> v2 = v;        // copy
```

Copy code

🔹 Capacity operations

Function	Meaning	Complexity
`size()`	number of elements	O(1)
`capacity()`	allocated slots	O(1)
`empty()`	true if size==0	O(1)
`reserve(n)`	pre-allocate memory	O(n)

Function	Meaning	Complexity
`shrink_to_fit()`	reduce capacity	O(n)

🔹 Element Access

Operation	Formula	Meaning
`v[i]`	random access	no bounds check
`v.at(i)`	safe access	throws error
`v.front()`	first element	
`v.back()`	last element	
`v.data()`	pointer to raw array	

🔹 Modifiers

Operation	Formula	Meaning
`push_back(x)`	add at end	
`pop_back()`	remove last	
`insert(pos,x)`	insert element	
`insert(pos,n,x)`	insert n copies	
`erase(pos)`	remove one	
`erase(l,r)`	remove range	
`clear()`	delete all	
`resize(n)`	resize	
`assign(n,x)`	replace all	
`swap(v2)`	swap	

✨ Tip
Insertion in middle = O(n)
Push_back = amortized O(1)

🔹 Iterators

cpp

```cpp
v.begin()
v.end()
v.rbegin()
v.rend()
```

Copy code

🟦 B. deque<T>

Double-ended dynamic array.

Supports fast:
✔ push_front
✔ push_back
❌ middle insert slow

🔹 Extra functions

Function	Meaning
`push_front(x)`	add at front
`pop_front()`	remove front
`front()`	first
`back()`	last

Access = O(1)

🟦 C. list<T>

Doubly-linked list

✔ fast insert/delete anywhere
❌ slow random access

🔹 Key operations

Function	Meaning
`push_front()`	
`push_back()`	
`pop_front()`	
`pop_back()`	
`insert(it,x)`	O(1)
`erase(it)`	O(1)
`remove(x)`	delete all x
`unique()`	remove duplicates
`sort()`	internal sort
`merge(list2)`	merge sorted
`splice()`	move nodes

🟦 D. forward_list<T>

Singly-linked list
(lower memory)

Supports:

```lua
push_front
insert_after
```

```
erase_after
remove
unique
sort
merge
```

No size()

Copy code

🟦 E. array<T,n>

Fixed-size static array with STL interface

2️⃣ ASSOCIATIVE CONTAINERS

(sorted, tree-based → O(log n))

Includes:

`set` , `multiset` , `map` , `multimap`

🟥 set<T>

Stores unique sorted elements.

🔹 Operations

Function	Meaning
`insert(x)`	add
`erase(x)`	remove
`find(x)`	iterator / end
`count(x)`	0 or 1
`lower_bound(x)`	≥ x
`upper_bound(x)`	> x

Function	Meaning
`equal_range(x)`	pair bounds
`size()`	
`clear()`	
`empty()`	

multiset<T>

Same as set BUT allows duplicates

`count(x)` may be >1

map<Key,Value>

Stores unique keys sorted.

Access

cpp

```cpp
m[key]        // inserts default if not exists
m.at(key)     // throws if missing
```

Copy code

Insert

cpp

```cpp
m.insert({key,val})
m.emplace(key,val)
```

Copy code

Iterating

cpp

```
for(auto &p: m)
   p.first, p.second
```

Copy code

🟥 multimap

Duplicates keys allowed

3️⃣ UNORDERED CONTAINERS

(hash-table → average O(1))

Includes:

`unordered_set`

`unordered_multiset`

`unordered_map`

`unordered_multimap`

🔹 Key Points

✔️ fast hashing
❌ no ordering
❌ unstable iteration order

Operations same as ordered except:

🚫 no lower_bound
🚫 no upper_bound

4️⃣ ITERATORS

Bridge between containers & algorithms

Types

- Input
- Output

- Forward
- Bidirectional
- Random access

Core Functions

cpp

```
begin()
end()
next(it)
prev(it)
advance(it,n)
distance(a,b)
```

Copy code

5️⃣ ALGORITHMS (💎 MOST IMPORTANT)

In `<algorithm>`

Sorting

cpp

```
sort(a,b)
stable_sort(a,b)
partial_sort
nth_element
```

Copy code

Searching

cpp

```
binary_search
lower_bound
upper_bound
equal_range
```

Copy code

Modifying

cpp

```
copy
move
fill
replace
transform
unique
remove
remove_if
rotate
reverse
swap
```

Numeric

Copy code

cpp

```
accumulate
inner_product
partial_sum
adjacent_difference
```

Copy code

6 UTILITY COMPONENTS

cpp

```
pair
tuple
make_pair
make_tuple
tie
swap
move
forward
optional (C++17)
```

```
variant   (C++17)
any       (C++17)
```

7️⃣ FUNCTORS

Copy code

Objects behaving like functions

Example:

cpp

```cpp
struct add {
  int operator()(int a,int b){ return a+b; }
};
```

Used in sort, maps, etc.

Copy code

8️⃣ LAMBDA EXPRESSIONS

Anonymous functions

cpp

```cpp
auto f = [](int x){ return x*x; };
```

Copy code

Capture:

cpp

```cpp
[x,&y]
```

Copy code

9️⃣ CONTAINER ADAPTORS

Built on other containers

---

stack

cpp

```
push
pop
top
size
empty
```

Copy code

queue

cpp

```
push
pop
front
back
```

Copy code

priority_queue

Max-heap default

🔟 ITERATOR ADAPTORS

cpp

```
back_inserter
front_inserter
inserter
istream_iterator
ostream_iterator
```

Copy code

1️⃣1️⃣ ALLOCATORS (Advanced)

Customize memory allocation.

Rare in CP.

🎯 SUPER-USEFUL COMPLEXITY SUMMARY

Container	Random Access	Insert Middle	Insert Ends
vector	O(1)	O(n)	back O(1)
deque	O(1)	O(n)	O(1) both
list	❌	O(1)	O(1)
set/map	O(log n)	O(log n)	O(log n)
unordered_*	O(1) avg	O(1) avg	O(1) avg

✅ IF YOU WANT NEXT

Tell me and I'll provide:

- ✔ Practice questions
- ✔ Interview checklist
- ✔ Visual diagrams
- ✔ One-page printable PDF
- ✔ Code examples for every function
- ✔ CP-focused shortcuts

Just say what format you like 🙂

One-page printable PDF

Here's your **one-page printable PDF cheat sheet** for C++ STL:

📄 Download the PDF

If you want it more detailed, colored, or formatted differently (A4/Letter, headings, tables, etc.), tell me and I'll update it 👍 ‹/›

